```
class Lab00 {
    static void Main() {
        Console.WriteLine("Ready");
    }
}
```

Human Computer Interaction

Lab 01

Learning Objectives

By the end of this lab, students will be able to:

01

Explain what **.NET** is.

02

Distinguish between C# and .NET.

03

Write simple C# programs

What is .NET?

.NET is a free, open-source, cross-platform development platform created by Microsoft

It provides:
A runtime for executing applications
A large collection of libraries
Development tools
Application frameworks
Support for multiple programming languages

.NET can be used to build:
Console applications
Web applications and APIs
Desktop applications
Mobile applications
Cloud services
Games and other applications

Platform, Runtime engine and Framework

Framework: A structured set of libraries, conventions, and reusable code that you build your application with. It gives you pre-built components, patterns (like MVC), and often dictates how you organize your code

Runtime (The Engine): The execution engine responsible for hosting, managing, and running your compiled C# code.

Core responsibilities:

- **JIT Compilation.**
- **Garbage Collection (GC).**
- **Type Safety & Exception Handling.**
- **Thread Management.**

Code that runs inside this runtime is called Managed Code.

Platform: Encompasses the entire end-to-end target environment needed to build, package, and execute an application. In the .NET world, "Platform" refers to both the hosting environment (OS/Hardware) and the unified .NET SDK/Tooling ecosystem.

Includes SDK & Tooling: Includes the dotnet CLI, compilers (csc), build engines (MSBuild), and package managers (NuGet).

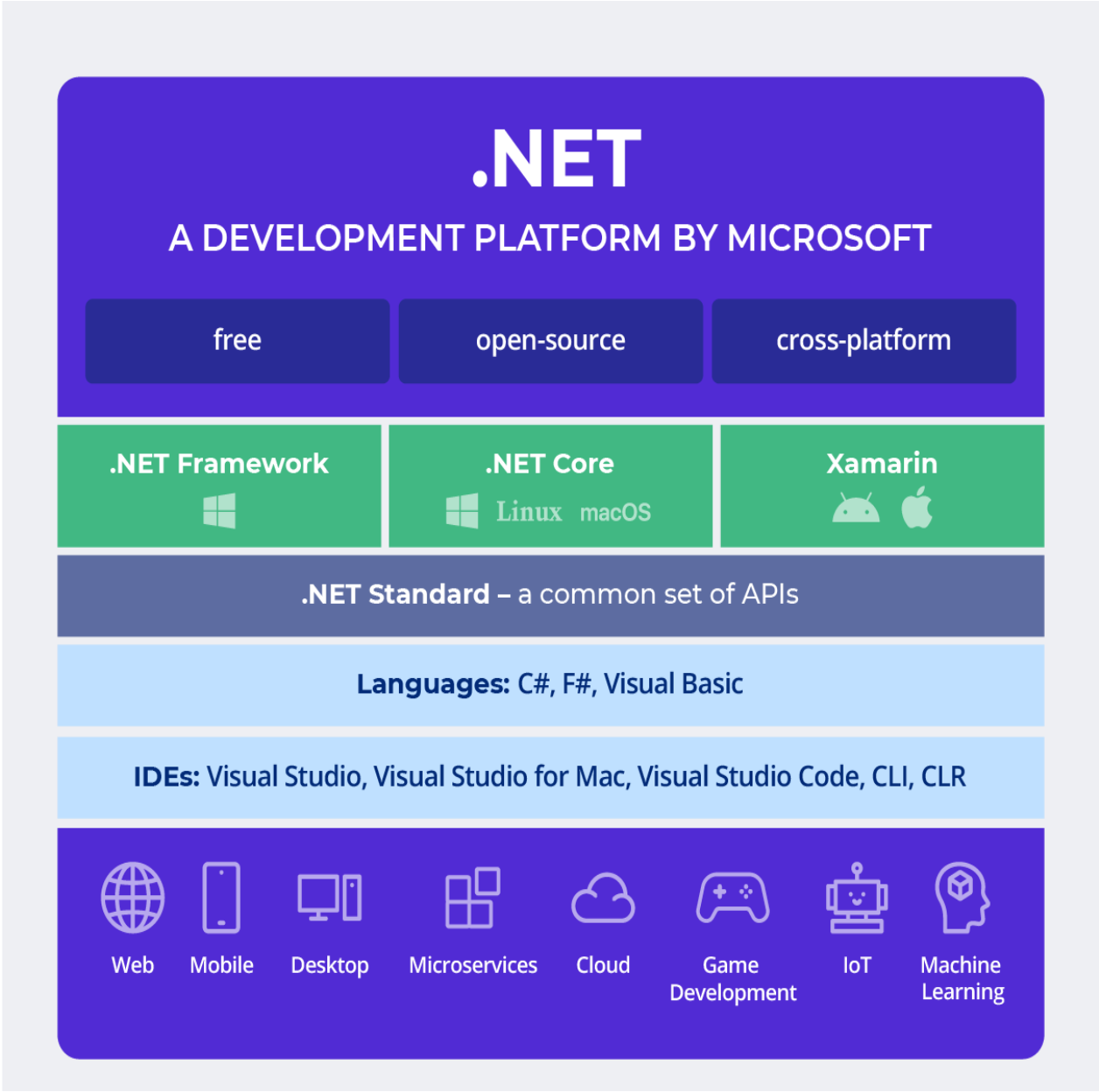
Software development kit (SDK):

SDK: Comprehensive, all-in-one package of development tools, libraries, documentation, and code samples that enables software engineers to build applications for a specific platform, hardware, operating system, or service.

Core Components:

Libraries and APIs: Pre-written code components, functions, and interfaces that abstract away low-level system interactions (e.g., handling network calls, accessing camera hardware, rendering graphics).
Compilers and Build Tools: Tools that translate human-readable source code into executable binaries or intermediate bytecodes
Debuggers and Profilers: Diagnostic tools used to step through code, inspect memory, analyze CPU usage, and locate runtime errors.
Code Samples and Documentation: Guides, reference manuals, and starter templates that demonstrate standard implementation patterns.
Emulators or Simulators: Virtual environments that allow testing code on targeted hardware or operating systems without requiring physical devices (e.g., Android Device Emulator).

.NET platform



Concepts review:

Q1. Which statement best describes .NET?
A. A programming language
B. An operating system
C. A development platform for building and running applications
D. A code editor

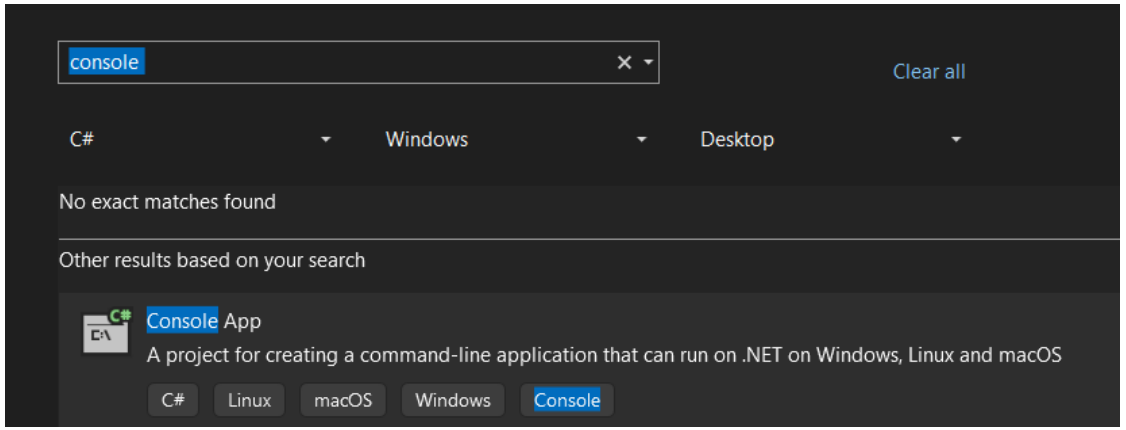
Q2. What is the main purpose of a framework?
A. To provide reusable components and structure for building applications
B. To execute machine instructions directly
C. To replace the operating system
D. To compile every programming language

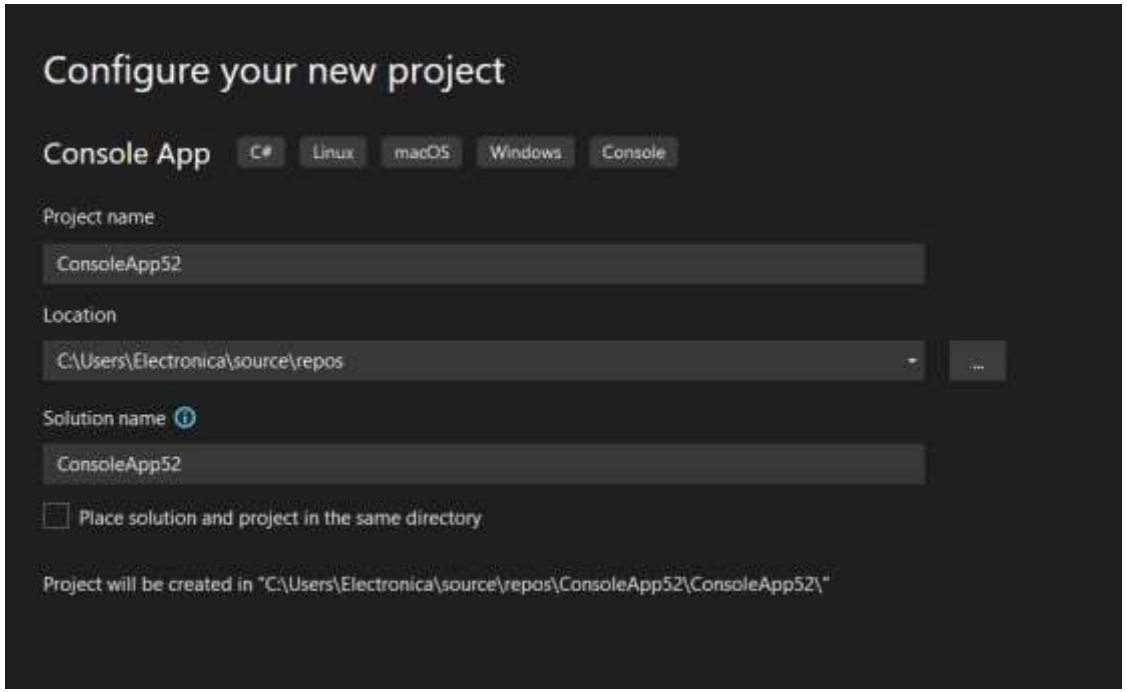
Q3. What is the main purpose of the .NET SDK?
A. To only run existing .NET applications
B. To provide tools needed to develop, build, and publish .NET applications
C. To act as the operating system for .NET applications
D. To store application data

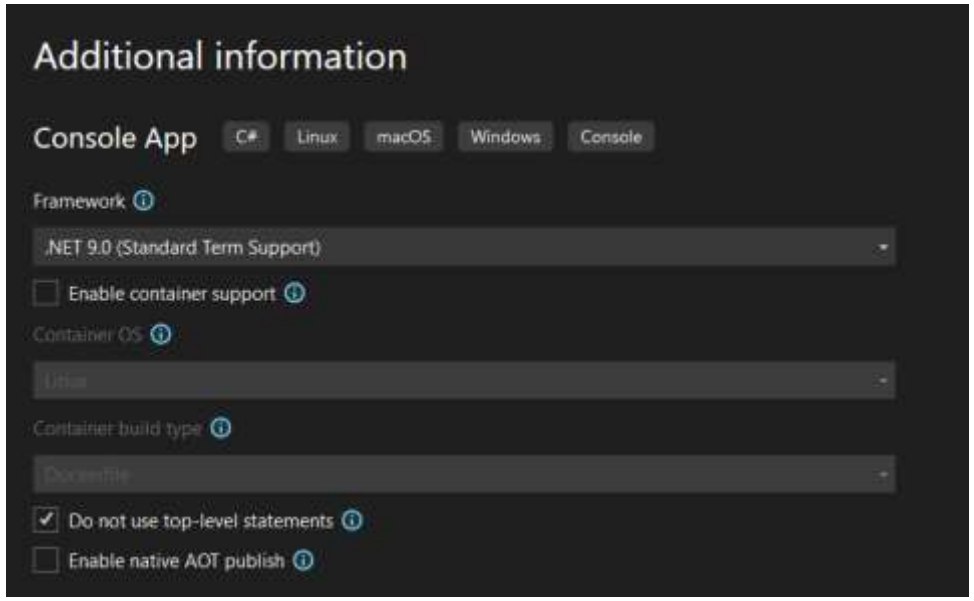
Q4. What is the main purpose of the .NET Runtime?
A. To create new projects
B. To provide development tools and project templates
C. To provide the environment required to execute .NET applications
D. To write C# source code

Q5. Which statement correctly describes the relationship between C#, .NET, SDK, and Runtime?
A. C# is a platform, .NET is a language, SDK runs applications, and Runtime creates projects
B. C# is a language, .NET is a platform, SDK provides development tools, and Runtime executes applications
C. C# is a framework, .NET is an operating system, SDK is a language, and Runtime is a compiler
D. C# and .NET are two different names for the same thing

How to create a C# console app from visual studio IDE:







How to create a C# console app from the CMD:

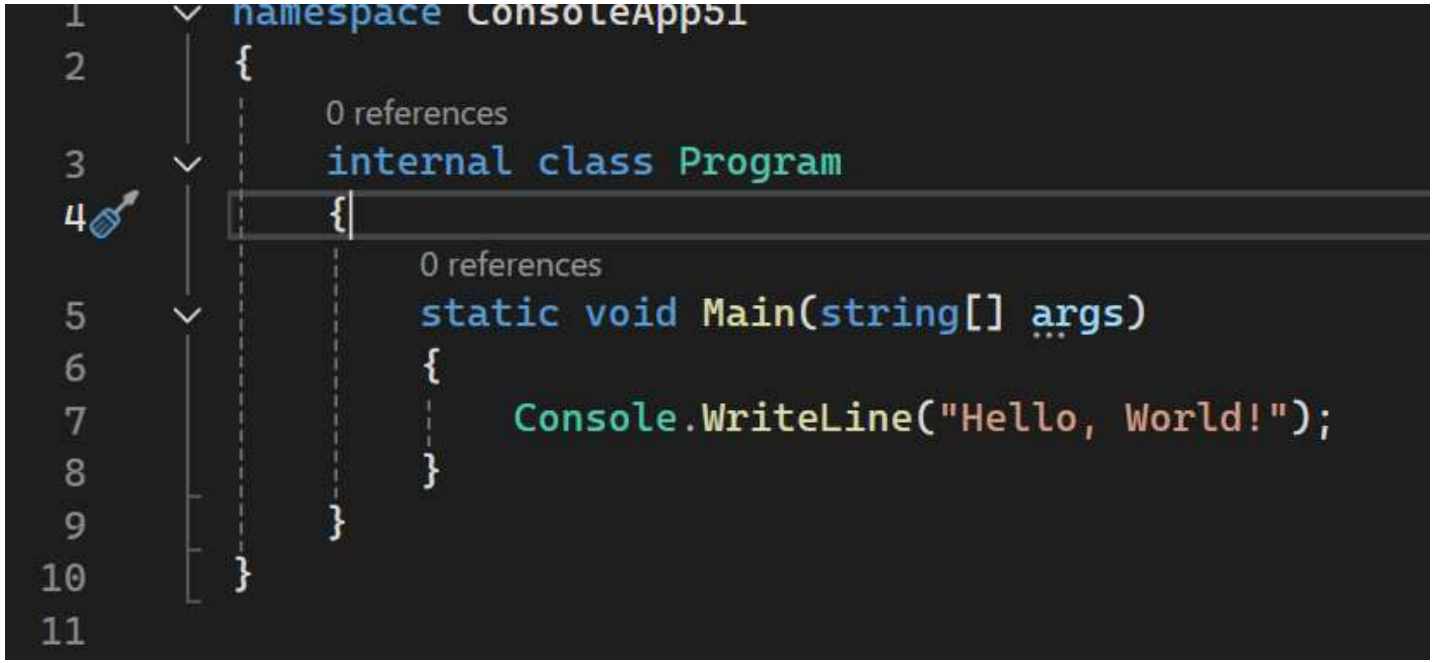
```
C:\Users\Electronica\Desktop>dotnet new console -n CSharpApplication
The template "Console App" was created successfully.

Processing post-creation actions...
Restoring C:\Users\Electronica\Desktop\CSharpApplication\CSharpApplication.csproj:
Restore succeeded.
```

```
C:\Users\Electronica\Desktop>cd CSharpApplication

C:\Users\Electronica\Desktop\CSharpApplication>dotnet run
Hello, World!
```

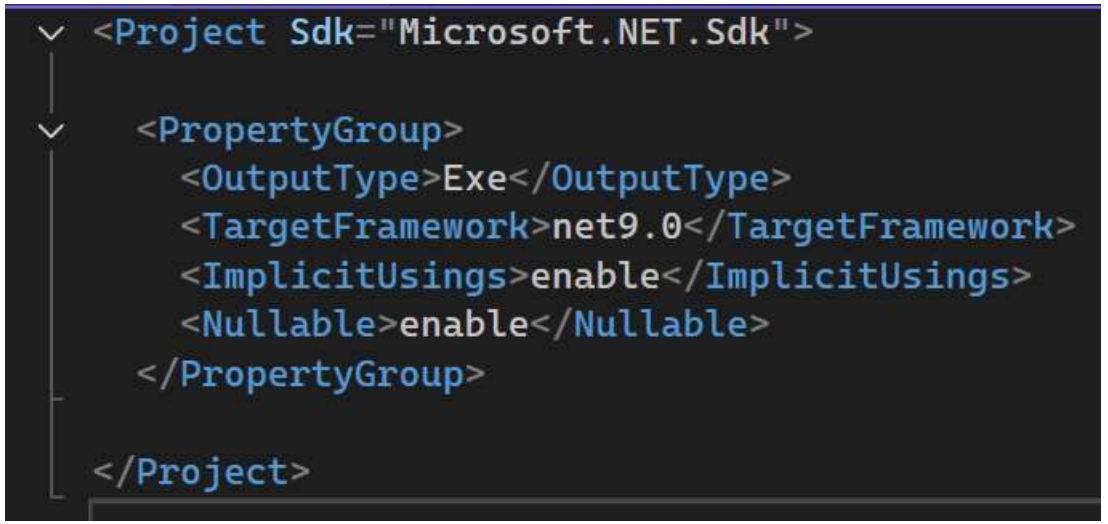
Sample project:



.csproj file

It contains project configuration such as:

- Target framework
- Project settings
- Dependencies/packages
- Build-related information



Data types in C#

C# is **statically typed**.

Data Types	Memory Size	Range
char	1 byte	-128 to 127
signed char	1 byte	-128 to 127
unsigned char	1 byte	0 to 255
short	2 byte	-32,768 to 32,767
signed Short	2 byte	-32,768 to 32,767
unsigned Short	2 byte	0 to 65,535
int	4 byte	-2,147,483,648 to 2,147,483,647
signed int	4 byte	-2,147,483,648 to 2,147,483,647
unsigned int	4 byte	0 to 4,294,967,295
long	8 byte	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
signed long	8 byte	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
unsigned long	8 byte	0 to 18,446,744,073709,551,615
float	4 byte	$1.5 * 10^{-45}$ to $3.4 * 10^{38}$ (7 Digit)
double	8 byte	$5 * 10^{-324}$ to $1.7 * 10^{308}$
decimal	16 byte	$-7.9 * 10^{28}$ to $7.9 * 10^{28}$

Variable definition and declaration

C# is statically typed which means it checks and enforces variable data types before the program runs, usually during the compilation step

DataType Identifier = Value;

```
static void Main(string[] args)
{
    //Variable definition and declaration
    int age;
    double price = 99.99;
    char grade = 'A';
    bool passed = true;
    string name = "Ali";
    age = 32;
}
```

The var keyword in C#:

Using var makes the compiler determines the type without explicitly mentioning it.

If you try to reassign the variable with a value of another type, you will get an error.

You can't use var with declarations only, the variable must be defined.

```
static void Main(string[] args)
{

    var x = 5;
    x = 18;
    x = "";
    var y;

}
```

```
var x = 5;
x = 18;
x = "";
var y;
```

readonly struct System.Int32
Represents a 32-bit signed integer.

Printing:

Write() → stays on the same line
WriteLine() → moves to the next line

String interpolation -using variables inside strings- can be achieved by using $ before the double quotations

```
static void Main(string[] args)
{

    Console.Write("Hello");
    Console.WriteLine("Hello world");

}
```

```
HelloHello world
```

```
0 references
static void Main(string[] args)
{
    string name = "Ali";
    Console.WriteLine($"Hello {name}");

}
```

```
Hello Ali
```

Reading input:

```
0 references
static void Main(string[] args)
{
    Console.Write("Enter your name: ");
    string name = Console.ReadLine();

}
```

Console.ReadLine() returns a **string**.

Which means when dealing with other data types we need to parse the input

```
Enter your name: ali
```

```
Console.Write("Enter a positive number: ");
number = int.Parse(Console.ReadLine());
number = Convert.ToInt32(Console.ReadLine());
```

Problem 1:

Personal Information

Write a program that asks the user for:

- Name
- Age
- City

Then prints:

Hello, my name is Ahmed.
I am 21 years old.
I live in Assiut.

Problem 2:

Simple Calculator
Read two numbers and display:
Sum
Difference
Product
Division
Remainder

If statement:

Notice:

- Parentheses around the condition
- { } define the block
- ; ends statements

```
0 references
static void Main(string[] args)
{
    int grade = 50;
    if (grade >= 85)
    {
        Console.WriteLine("Excellent");
    }
    else if (grade >= 70)
    {
        Console.WriteLine("Very Good");
    }
    else if (grade >= 50)
    {
        Console.WriteLine("Pass");
    }
    else
    {
        Console.WriteLine("Fail");
    }
}
```

Switch:

Key points:

- case defines a possible value.
- break exits the switch.
- default handles unmatched values.



```
0 references
static void Main(string[] args)
{
    int day = 2;

    switch (day)
    {
        case 1:
            Console.WriteLine("Saturday");
            break;

        case 2:
            Console.WriteLine("Sunday");
            break;

        case 3:
            Console.WriteLine("Monday");
            break;

        default:
            Console.WriteLine("Invalid day");
            break;
    }
}
```

For loop:

Use a for loop when the number of iterations is known or can be controlled with a counter.

for (initialization; condition; update)

```
0 references
static void Main(string[] args)
{
    for (int i = 1; i <= 5; i++)
    {
        Console.WriteLine(i);
    }
}
```

While loop:

A while loop repeats **as long as the condition is true**.

```
static void Main(string[] args)
{
    int i = 1;

    while (i <= 5)
    {
        Console.WriteLine(i);
        i++;
    }
}
```

Do while loop:

do...while loop executes its body **at least once**, because the condition is checked after the body.

```
static void Main(string[] args)
{
    int number;

    do
    {
        Console.Write("Enter a positive number: ");
        number = int.Parse(Console.ReadLine());

    } while (number <= 0);
}
```

For vs While vs do while:

Loop	When to use
for	Number of iterations is known
while	Repeat while a condition is true
do...while	Execute at least once, then check condition

Problem 3:

Loop Problem — Number Analysis
Write a C# program that repeatedly asks the user to enter an integer.
The program should:
Continue reading numbers until the user enters **0**.
For every number entered:

- Determine whether it is **even or odd**.
- Add it to the appropriate total.

When 0 is entered, display:

- The total number of values entered.
- The sum of even numbers.
- The sum of odd numbers.

Array Creation:

- Array indexes start at **0**.
- We use .length to get the array length
- For example:
- Numbers.Length

```
0 references
static void Main(string[] args)
{
    int[] grades = { 80, 75, 90 };
    int[] numbers = new int[5];
    int[] numbers2 = { 10, 20, 30, 40, 50 };

    Console.WriteLine(numbers[0]); // 10
    Console.WriteLine(numbers[3]); // 40

}
```

Looping through arrays:

- For vs foreach
- for→ Useful when you need the index.
- foreach→ Useful when you only need each value.

```
int[] numbers = { 10, 20, 30, 40, 50 };

foreach (int number in numbers)
{
    Console.WriteLine(number);
}
```

```
int[] numbers = { 10, 20, 30, 40, 50 };

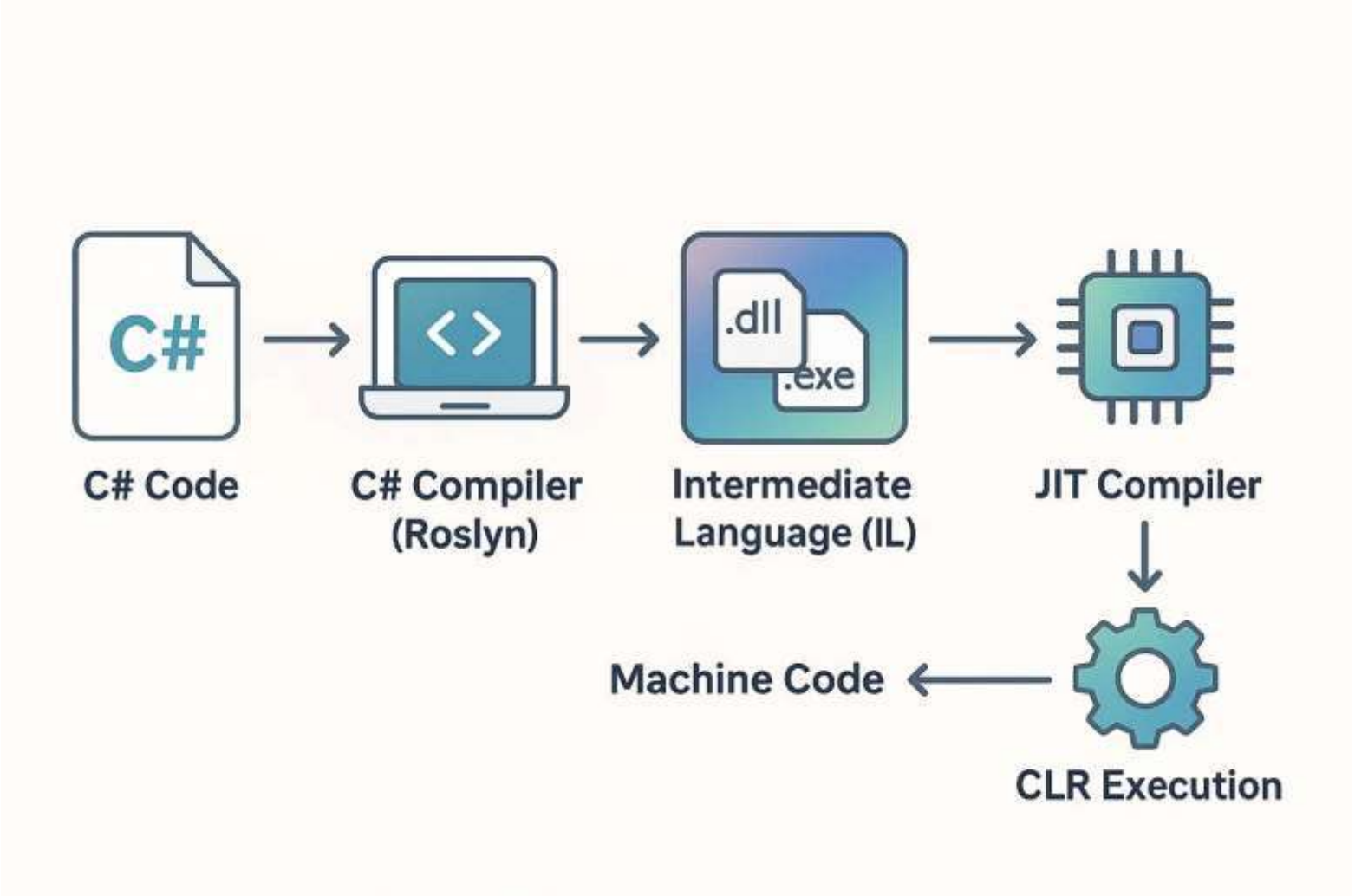
for (int i = 0; i < numbers.Length; i++)
{
    Console.WriteLine(numbers[i]);
}
```

Problem 4:

Write a C# program that:
Creates an integer array of size 5.
Reads 5 numbers from the user.
Prints all numbers.
Calculates and prints:
- Sum
- Average
- Maximum
- Minimum

Counts how many numbers are even.

C# compilation process:



HCI Facebook group

